



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026-27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 10

Team No.: 4

Project Title: Process Termination Messages – A Linux Utility for Reporting Process Exit Status

Team Members

Roll Number	Student Name	Signature
2520090139	A.Shashank Reddy	
2520090071	D.Vignesh Kumar	
2520090062	Y.Sharath	

1. Abstract

Process termination in Linux can occur in several ways — normal exit with a status code, termination by an unhandled signal, or abnormal exit with a core dump — yet most simple programs report little of this to the user. This project, Process Termination Messages, builds a Linux utility that creates child processes using `fork()` and `execvp()`, monitors their completion with `wait()/waitpid()`, and decodes the returned status using the POSIX macros `WIFEXITED`, `WEXITSTATUS`, `WIFSIGNALED`, `WTERMSIG` and `WCOREDUMP`.

The utility prints clear, human-readable termination messages reporting the process ID, the command executed, the exit code or terminating signal, and whether a core dump was produced. It also demonstrates asynchronous notification through the `SIGCHLD` signal handled via `sigaction()`, reinforcing core Operating Systems concepts such as process creation, the parent-child relationship, and kernel-level exit-status communication. The expected outcome is a working command-line tool that launches and tracks processes, giving practical experience with POSIX process-management system calls in Linux.

2. Problem Statement

Standard Linux shells and simple programs typically report only whether a child process succeeded or failed, without clearly distinguishing between a normal exit, termination by a signal such as `SIGSEGV` or `SIGKILL`, or an abnormal exit that produced a core dump. Developers need this finer-grained information to debug crashes and build robust process-supervision tools, but correctly decoding the `wait()` status word using POSIX macros is non-trivial for beginners. This project addresses the need for a reliable Linux utility that accurately captures and reports the exact cause and manner of process termination.

3. Objectives

- Design a parent process that creates one or more child processes using `fork()` and executes target programs using `execvp()`.
- Monitor child termination using `wait()` and `waitpid()`, and decode the status using `WIFEXITED`, `WEXITSTATUS`, `WIFSIGNALED`, `WTERMSIG` and `WCOREDUMP`.
- Generate formatted, human-readable termination messages reporting PID, command name, exit code/signal, and core-dump status.
- Handle asynchronous termination notifications for multiple concurrent child processes using the `SIGCHLD` signal and a `sigaction()`-based handler.

4. Proposed Methodology

The system will be developed in C on a Linux/Ubuntu platform. The parent process spawns one or more child processes using `fork()`; each child uses `execvp()` to run a specified command. The parent then waits for its children either synchronously, in a loop of `waitpid(-1, &status, 0)`, or asynchronously, by registering a `SIGCHLD` handler with `sigaction()` so termination is reported as soon as the kernel delivers the signal.

Once a child's status is obtained, the program decodes it using `WIFEXITED()/WEXITSTATUS()` for normal exits and `WIFSIGNALED()/WTERMSIG()/WCOREDUMP()` for signal-based termination, converts signal numbers to readable names with `strsignal()`, and prints a formatted message with the PID, command, timestamp, and outcome. The utility will be tested against processes that exit normally, that are killed by signals such as `SIGSEGV` and `SIGKILL`, and that dump core, to validate correct message generation in every case.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
<code>fork()</code>	Creates a new child process to run and monitor the target program.
<code>execvp()</code> (exec family)	Replaces the child process image with the program whose termination is being tracked.
<code>wait()</code> / <code>waitpid()</code>	Suspends the parent until a child terminates and retrieves its exit status.
<code>WIFEXITED()</code> , <code>WEXITSTATUS()</code> , <code>WIFSIGNALED()</code> , <code>WTERMSIG()</code> , <code>WCOREDUMP()</code>	POSIX macros used to decode the <code>wait()</code> status word and build the termination message.
<code>sigaction()</code> / <code>SIGCHLD</code>	Registers a handler that is invoked asynchronously when a child process terminates.
<code>strsignal()</code> , <code>getpid()</code> / <code>getppid()</code>	Converts signal numbers to readable names and identifies process IDs for reporting.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520090139	A.Shashank Reddy	
2520090071	D.Vignesh Kumar	
2520090062	Y.Sharath	

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Development, compilation, and execution environment.
C / C++	Implementation language for process creation and system-call programming.
GCC / Make	Compiling and building the source code.
GDB	Debugging process behaviour and signal handling.
VS Code / vim	Source code editing and project development.

8. Expected Outcome

A working Linux command-line utility that spawns one or more processes and, upon their termination, prints a clear, structured message stating whether the process exited normally (with its exit code) or was terminated by a signal (naming the signal and indicating whether a core dump was produced).

The project will demonstrate correct use of `fork()`, `execvp()`, `wait()/waitpid()`, and `SIGCHLD`-based asynchronous handling, and will produce sample logs of termination messages for both normal and abnormal process exits, validating a practical understanding of Linux process lifecycle management.

9. GitHub Repository

GitHub Repository Link: https://github.com/vigneshkumar-stack/Process-Termination-Messages_OSSP_04

Repository Created: _____

☒ Yes ☐ No

Faculty Name and Signature

Faculty Name: _____ *M. Jaghazeth*

Signature: _____ *[Signature]*

Remarks: _____

Faculty Use

Project Approved: _____

☒ Yes ☐ No

Date: 9/2/2016

Faculty Signature: [Signature]